

CloudSpend

Cloud Billing ETL & Cost Insights

A complete, plain-English guide to the project

What you will learn

What the project is • the technologies and why • where the data came from • how it is stored (star schema in SQLite) • how to view it in DBeaver • every command to run it • how Streamlit connects • how anomaly detection and forecasting work • interview Q&A.

github.com/Chaitanya060/CloudSpend

1. What is this project (in one paragraph)

CloudSpend is a small **cloud-cost analytics platform** (a "FinOps" tool). Companies that use AWS, Azure and Google Cloud get huge, messy monthly bills. CloudSpend takes that raw billing data, **cleans it, organises it into a database**, and then **shows useful insights**: which services cost the most, whether spend is trending up, whether any day had an unusual cost **spike**, and a **forecast** of next month's spend — all on an interactive web dashboard.

The flow is the classic data-engineering pipeline: **raw data** → **ETL** → **database (star schema)** → **analytics** → **dashboard**. "ETL" means **Extract, Transform, Load** — read the data, clean it, and store it.

The big picture (data flow)

```
datasets/focus_sample.csv          (real cloud billing data)
|
v
[ src/etl.py ]  EXTRACT -&gt; read the CSV
|              TRANSFORM -&gt; clean dates, handle credits,
|                  resolve provider + category
|              LOAD -&gt; aggregate to daily, write to database
v
[ SQLite database: data/cloudspend.db ]  &lt;- STAR SCHEMA
|    fact_cost + dim_date + dim_account + dim_service
v
[ analytics ]  anomaly.py (spikes) + forecast.py (prediction)
|
v
[ dashboard/app.py ]  Streamlit web app (charts + tables)
```

2. Technologies used (and why)

Every tool in the project and the reason it is there:

Technology	What it is	Why it is used here
Python	General-purpose programming language	The language the whole pipeline is written in
Pandas	Python library for tables of data (DataFrames)	Reads the CSV, cleans it, and aggregates to a daily total
NumPy	Numerical computing library	Maths used by the forecast and stats
scikit-learn	Machine-learning library	The Linear Regression used for the 30-day forecast
SQLAlchemy	Database toolkit for Python	Talks to the database; lets the SAME code use SQLite or MySQL
SQLite	A file-based database (built into Python)	Stores the star schema; no server to install
MySQL	A server-based database (optional)	Production option; enabled with one setting
Streamlit	Turns Python scripts into web apps	Builds the interactive dashboard
Plotly	Charting library	Draws the bar charts, pie/donut and line charts

Technology	What it is	Why it is used here
Git / GitHub	Version control + code hosting	Stores the code; Streamlit Cloud deploys from it
AWS S3/Lambda	Cloud storage + serverless compute	Optional event-driven layer (reference code)

3. Where the data came from

This is the most important question for an interview, so know it well.

The data is the **FinOps Foundation's official FOCUS 1.0 sample dataset**. It is a real, publicly published cloud billing dataset that lives in the project at **datasets/focus_sample.csv** (10,000 rows).

Fact	Detail
Source	FinOps Foundation - FOCUS-Sample-Data (GitHub), FOCUS 1.0
What is FOCUS	FinOps Open Cost & Usage Specification - the open STANDARD format that AWS, Azure and GCP all export their bills to
Clouds covered	AWS, Microsoft Azure, Oracle (real multi-cloud)
Rows / period	10,000 rows, September 2024
Real details	Real service names (Amazon EC2, Azure SQL...), regions, tags, and even credits/refunds (negative costs)

How it was taken: it was downloaded as a CSV file from the FinOps Foundation's public GitHub repository and saved into the project's **datasets/** folder. Nothing connects to a live AWS account — the file is self-contained, which is why the project runs offline.

Interview one-liner: "I used the FinOps Foundation's FOCUS 1.0 sample dataset — FOCUS is the open billing standard that AWS, Azure and GCP all export to — so my pipeline ingests the exact schema a real FinOps team works with, across three clouds."

Note: the amounts are small (about \$151 total) because it is a published sample. The format is real; only the volume is small. The pipeline handles large volumes identically.

4. How the data is stored (the star schema)

After cleaning, the data is stored in a **star schema** — the standard way analytics databases are organised. There is one central **fact** table (the numbers) surrounded by **dimension** tables (the descriptions). It is called a "star" because the diagram looks like one.

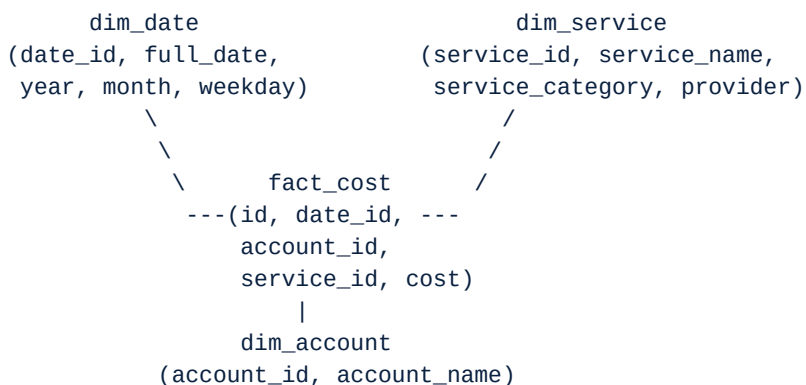


Table	Rows	What it holds
fact_cost	748	One row per day + account + service, with the summed cost (the 'measure')
dim_date	30	Calendar info for each date (year, month, weekday)
dim_account	3	The billing accounts (SunBird on AWS, SunBird on Azure, an Oracle account)

Table	Rows	What it holds
dim_service	63	Each cloud service, plus its provider (AWS/Azure/Oracle) and category (Compute, Storage...)

All four tables live inside a single file: **data/cloudspend.db**. That one file *is* the SQLite database — there is no separate database server running.

Why a star schema and not one big table? It removes repetition (a service's name and provider are stored once in dim_service, not on every row), and it makes analytical queries — "total by provider", "top services" — simple, fast joins. This design is what signals real *data engineering* rather than just scripting.

5. How to run the project (commands)

Open a terminal **inside the CloudSpend folder** and run these in order. (On Windows use PowerShell or the terminal in VS Code.)

Step 1 - Install the required libraries (once)

```
pip install -r requirements.txt
```

This reads requirements.txt and installs Pandas, Streamlit, etc.

Step 2 - Run the pipeline (build the database)

```
python run_pipeline.py
```

This runs the ETL: it reads the CSV, cleans it, creates the star schema inside **data/cloudspend.db**, then prints the detected anomalies and the forecast. Run this whenever you want to (re)build the database.

Step 3 - Launch the dashboard

```
python -m streamlit run dashboard/app.py
```

This starts the web app and opens it in your browser at **http://localhost:8501**. Leave the terminal open while you use it; press Ctrl+C in the terminal to stop it.

Optional - run individual stages

```
python -m src.etl                # just the ETL / load
python -m src.analytics.anomaly  # print detected anomalies
python -m src.analytics.forecast # print the forecast summary
```

Optional - use MySQL instead of SQLite

PowerShell (Windows):

```
$env:CLOUDSPEND_DB = "mysql"
$env:MYSQL_PASSWORD = "yourpassword"
python run_pipeline.py
```

Because everything goes through SQLAlchemy, no code changes are needed — the same tables are created in MySQL.

6. Viewing the data in DBeaver

DBeaver is a free tool to browse databases visually. Here is how to open your SQLite file and see the tables.

Step	What to do
1	Open DBeaver. Click Database menu -> New Database Connection (or the plug icon top-left).
2	In the list, choose SQLite, then click Next.
3	Click 'Open' / browse next to 'Path' and select the database file:
4	If DBeaver asks to download the SQLite driver, click Download.
5	Click 'Test Connection' -> it should say Connected. Click Finish.
6	In the left panel expand: your connection -> Tables. You will see fact_cost, dim_date, dim_account, dim_service.
7	Double-click any table, then open the 'Data' tab to see the rows.

The exact file path to select in step 3:

C:\Users\chait\OneDrive\Desktop\CloudSpend\data\cloudspend.db

If the file is not there, run 'python run_pipeline.py' first - the database file is created by the pipeline (it is not stored in Git).

Try a query in DBeaver (SQL Editor)

Click your connection, then SQL Editor -> New SQL Script, paste this, and run it (Ctrl+Enter) to see total spend per cloud provider:

```
SELECT s.provider, ROUND(SUM(f.cost), 2) AS total_cost
FROM fact_cost f
JOIN dim_service s ON f.service_id = s.service_id
GROUP BY s.provider
ORDER BY total_cost DESC;
```

7. How Streamlit connects to the data

Streamlit is just Python. The dashboard script (**dashboard/app.py**) does not hold any data itself — it **reads from the same SQLite database** the pipeline built. The chain is:

```
dashboard/app.py
- &gt; src/queries.py      (runs SQL: joins fact_cost to the dimensions)
- &gt; src/db.py           (SQLAlchemy engine - &gt; data/cloudspend.db)
- &gt; src/analytics/...   (anomaly + forecast, computed on the fly)
- &gt; Plotly              (turns the results into charts)
```

The key pieces, in plain English:

- **The engine (src/db.py):** SQLAlchemy opens a connection to the database file. One function, `get_engine()`, is used everywhere, so switching to MySQL changes nothing else.
- **The queries (src/queries.py):** small functions that run SQL joining `fact_cost` to the dimension tables and return the answer as a Pandas DataFrame (e.g. 'spend by provider', 'top services').
- **Self-building (src/bootstrap.py):** when the dashboard starts, it checks whether the database exists. If not (for example on a fresh Streamlit Cloud deploy) it runs the ETL automatically, so the app is never empty.
- **Caching (@st.cache_data):** Streamlit remembers the query results for a short time so the page stays fast and does not re-read the database on every click.
- **Charts:** the DataFrames are handed to Plotly, which draws the bars, donut and lines you see on the page.

Deployed version: the app is also hosted on **Streamlit Cloud**, which runs the exact same code straight from the GitHub repo. Every time you push to GitHub, Streamlit Cloud redeploys automatically.

8. How anomaly detection & forecasting work

Anomaly detection (finding cost spikes)

For each service, the code looks at the last 7 days of cost and works out the **average** and how much the cost normally **wobbles** (the standard deviation). If a day's cost is far above normal — higher than *average + 2 standard deviations* — it is flagged as an anomaly. A **materiality floor** is added so tiny (sub-cent) amounts are ignored and only meaningful spikes are reported. This mirrors how real FinOps alerting avoids noise.

Forecasting (predicting next month)

The code fits a straight line (**Linear Regression** from scikit-learn) through the daily total cost, then extends that line **30 days** into the future to estimate upcoming spend. It reports the trend (dollars per day) and the projected 30-day total.

9. Likely interview questions & answers

Question	Answer you can give
What is this project?	An ETL and analytics pipeline for cloud billing data: it ingests real multi-cloud bills, models them into a star schema, and surfaces cost insights - anomaly detection and a forecast - on a Streamlit dashboard.
Where did the data come from?	The FinOps Foundation's FOCUS 1.0 sample dataset - the open billing standard AWS, Azure and GCP export to - covering AWS, Azure and Oracle.
How do you store it?	In a star schema (fact_cost plus dim_date, dim_account, dim_service). Locally it is a SQLite file; the same SQLAlchemy code also targets MySQL.
Why a star schema?	It avoids repetition and makes analytical joins simple and fast - the standard dimensional model for a warehouse.
How does anomaly detection work?	Per service, a 7-day rolling mean plus 2 standard deviations, with a materiality floor so trivial spend does not cause false alarms.
How does the dashboard get the data?	Streamlit reads the same database through SQLAlchemy queries, computes the analytics on the fly, and renders them with Plotly.
How is it deployed?	On Streamlit Cloud, straight from GitHub; it self-builds its database on first load so it is never empty.
How would this scale to production?	Switch the backend to MySQL/RDS via one env var, and use the included S3 -> Lambda -> RDS handler so new billing files are processed automatically (event-driven).

10. Mini glossary

Term	Plain meaning
ETL	Extract, Transform, Load - read data, clean it, store it
FinOps	Cloud financial management - controlling cloud spend
FOCUS	Open standard format for cloud billing data
Star schema	A fact table (numbers) surrounded by dimension tables (descriptions)
Fact table	The table with the measurements - here, cost
Dimension	A table describing the facts - date, account, service
Grain	The level of detail of a fact row - here, one day x account x service
Anomaly	An unusual value - here, a cost spike
Forecast	A prediction of future values - here, next 30 days of spend
SQLite	A database that is just a single file, built into Python
SQLAlchemy	Python library that talks to databases
Streamlit	Turns a Python script into a web dashboard